

TerraMarket — Agricultural Marketplace

Software Requirements Specification

Course Code: INT 221

Course Name: MVC PROGRAMMING

Student Names: AADARSH SHARMA ,SHUBHAM ,SHIVAM

Registration Numbers: 12314695, 12314083,12314517

Prepared for
Continuous Assessment 3
Spring 2025

Revision History

Date	Version	Description	Author
28-04-2026	1.0	Set up the base Laravel project with login/register, product listings, and basic order flow	Shubham
12-05-2026	2.0	Built the 3D panorama section with Three.js, added cart feature, swapped placeholder images with real ones	Aadarsh Sharma
14-05-2026	3.0	Got everything working on Vercel with Neon Postgres — had to fix a bunch of serverless issues along the way	Shivam

1. Introduction

This document lays out the software requirements for TerraMarket, an online marketplace we built for agricultural trading. The whole idea behind the project was pretty straightforward — farmers in India often struggle to find reliable suppliers for things like seeds, fertilizers, and tools. We wanted to build a platform where they could browse products, compare prices, and place orders without having to rely on middlemen. The app is built on Laravel 12 using the MVC pattern, with a Neon PostgreSQL database on the backend and a Vercel serverless setup for hosting. We also threw in a 3D panoramic field view using Three.js because, honestly, it just made the landing page look way more impressive.

1.1 Purpose

We wrote this SRS mainly to document what the system does, how it works, and what decisions we made along the way. It is meant for our professors evaluating the project, but it would also be useful for any developer who picks up the codebase later. Think of it as a mix between a technical spec and a project diary — we have tried to cover everything from the database schema to the deployment quirks we ran into on Vercel.

1.2 Scope

Here is what TerraMarket can do right now:

- Users can sign up as either a Farmer or a Supplier — each role sees different parts of the app
- Suppliers get a form to list their products under four categories: seeds, fertilizers, manures, and tools
- Farmers can browse the full marketplace and filter by category
- There is a shopping cart that lives in the server session — you can add stuff, change quantities, or remove items
- Checkout creates order records and automatically reduces the product stock
- The dashboard shows your recent orders, quick stats, and shortcuts depending on your role
- The landing page has this cool 3D panoramic view built with Three.js and WebGL shaders
- We used GSAP for the loading animation — it does this kinetic text reveal thing that looks pretty slick
- The whole thing runs on Vercel as a serverless PHP function talking to Neon Postgres in the cloud

What we did NOT build (yet): real payment processing, live chat between buyers and sellers, or a mobile app.

1.3 Definitions, Acronyms, and Abbreviations

Term	What it means
MVC	Model-View-Controller — the architectural pattern Laravel follows
SRS	Software Requirements Specification (this document)
Laravel	The PHP framework we used, version 12
Three.js	A JavaScript library for doing 3D graphics in the browser
GSAP	GreenSock Animation Platform — handles all the fancy animations on the landing page
Neon	The cloud Postgres provider we picked because it has a generous free tier
Vercel	Where the app is hosted — it runs PHP as a serverless function
CRUD	Create, Read, Update, Delete — the basic database operations
Vite	The frontend bundler that compiles our JS and CSS
SNI	Server Name Indication — a TLS thing that caused us headaches with Neon connections
WebGL	The browser API that lets Three.js render 3D stuff using the GPU

1.4 References

- Laravel docs — <https://laravel.com/docs/12.x> (we referred to this constantly)
- Three.js docs — <https://threejs.org/docs/>
- Vercel deployment guide — <https://vercel.com/docs>
- Neon Postgres docs — <https://neon.tech/docs> (especially the SNI troubleshooting page)
- IEEE Std 830-1998 — the standard this SRS format is based on

1.5 Overview

The rest of this document is split into sections. Section 2 gives you the big picture — what the product does, who uses it, and what limits we worked within. Section 3 gets into the nitty-gritty: every feature, every interface, every non-functional requirement. Section 4 has our data flow diagrams. After that, sections 5 through 11 have the project links and whatever proof documents are needed for the submission.

2. General Description

2.1 Product Perspective

TerraMarket is a standalone web app — it is not plugging into some bigger system. We built it from scratch using Laravel 12. The architecture is pretty standard: Blade templates handle the frontend (styled with Tailwind via CDN), Laravel controllers manage the business logic, and Eloquent ORM talks to a PostgreSQL database hosted on Neon. The interesting part is the deployment — instead of a traditional server, we are running PHP as a serverless function on Vercel. That came with its own set of challenges, like dealing with a read-only filesystem and figuring out how to make Laravel's cache and session system work in /tmp.

2.2 Product Functions

In plain terms, here is what users can do with TerraMarket:

- Sign up and log in — the registration form asks whether you are a farmer or supplier, and the app adjusts what you see based on that
- List products (suppliers only) — fill out a form with the product name, category, price, stock count, and an image link
- Browse the marketplace — see everything that is listed, or narrow it down by category using the filter tabs
- Use the cart — add items, bump up quantities, remove stuff, and then check out when you are ready
- Place orders — pick a payment method (we support Cash on Delivery, UPI, Net Banking, and Card as options), submit, and the stock updates right away
- Check your dashboard — it shows your order history, some stats, and quick links to common actions
- Experience the 3D view — the landing page has an interactive panoramic scene that responds to mouse movement

2.3 User Characteristics

We designed the app for two kinds of people. First, farmers — these are folks who might not be super tech-savvy, so we kept the UI clean with big buttons, readable fonts, and not too many steps to get things done. The earth-tone color scheme was a deliberate choice to feel familiar and trustworthy rather than flashy. Second, suppliers — agricultural dealers who want to list their inventory online. They need to be comfortable with basic form filling (product name, price, etc.), but nothing more complicated than that.

2.4 General Constraints

- Vercel gives us a 300 MB limit on the serverless bundle, which is why we had to compress images aggressively
- The filesystem is read-only on Vercel — so we redirect all of Laravel’s cache, session, and log paths to /tmp
- Neon’s free tier has connection limits, plus there is a cold-start delay when the database has been idle for a while
- The PHP runtime on Vercel (vercel-php 0.9.0) does not have every extension you would get on a normal server
- We are using cookie-based sessions because file or database sessions do not play well with serverless
- All images had to be kept under about 2 MB each to fit within deployment limits

2.5 Assumptions and Dependencies

- Users have a reasonably modern browser — Chrome, Firefox, Safari, or Edge from the last couple of years
- Neon and Vercel stay up and keep offering their free tiers (if either goes down, the app goes down)
- We are pulling some product images from Unsplash, so that service needs to remain accessible
- WebGL support is needed for the 3D panorama, but the rest of the site works fine without it
- This is an online-only app — there is no offline mode or local storage fallback

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The app has six main screens:

- Landing page — first thing users see. Has a kinetic text preloader, a hero section with a floating glass card, four featured products, and the 3D scenery strip at the bottom
- Login and Register pages — clean forms with real-time validation. Registration includes a role picker for farmer vs supplier
- Dashboard — different for each role. Farmers see their order history and spending. Suppliers see their listed products and sales
- Marketplace — a grid of all products with category filter tabs at the top. Each card shows the image, price, stock, and an Add to Cart button
- Cart page — lists everything you have added, lets you change quantities or remove items, shows subtotals and grand total, and has a checkout section
- Add Product form — only accessible to suppliers. Fields for product name, category dropdown, description, price, stock, and image URL

3.1.2 Hardware Interfaces

Nothing special here — the app runs in a browser, so any device with internet access works. A GPU helps with the Three.js 3D view, but the site still functions perfectly without one; you just see a flat background instead.

3.1.3 Software Interfaces

Software	Version	What we use it for
Laravel	12.58.0	Backend framework — handles routing, controllers, models, everything
PHP	8.5.2	The language Laravel runs on
PostgreSQL via Neon	16.x	Our cloud database — stores users, products, and orders
Three.js	0.184.0	Powers the 3D panoramic view on the landing page
Vite	7.0.7	Bundles our JavaScript and CSS during the build step
vercel-php	0.9.0	The runtime that lets Vercel execute PHP code as serverless functions

3.1.4 Communications Interfaces

Everything goes over HTTPS. The browser talks to Vercel's edge network, which routes requests to our PHP function. Database connections use SSL (we set `sslmode=require` in the config) and we had to add an explicit endpoint ID in the connection string because Vercel's PHP build does not support SNI natively. Every form submission includes a CSRF token that Laravel checks automatically.

3.2 Functional Requirements

3.2.1 User Registration and Login

- Introduction: People need to create an account before they can use the marketplace. During signup, they pick whether they are a farmer or a supplier.
- Inputs: Name, email, password, password confirmation, and role (farmer or supplier).
- Processing: We validate everything server-side, hash the password with `bcrypt`, save the user to Postgres, and set up an authenticated session cookie.
- Outputs: On success, they land on their dashboard. On failure, they see specific error messages next to the fields that had problems.
- Error Handling: If someone tries to register with an email that is already taken, they get a clear message. Passwords have minimum length rules. Expired CSRF tokens return a 419 page.

3.2.2 Product Listing (Supplier Only)

- Introduction: Suppliers fill out a form to put their products on the marketplace. Farmers cannot access this feature.
- Inputs: Product name, category (picked from a dropdown with four options), description, price, stock count, and optionally an image URL.
- Processing: The controller checks that the logged-in user actually has the supplier role, validates all the fields, and then creates a product record linked to that user.
- Outputs: The supplier gets redirected to their My Products page with a success toast. The new product shows up in the marketplace immediately.
- Error Handling: If a farmer somehow tries to hit this endpoint, they get bounced back to the marketplace with an error. Missing or invalid fields show per-field validation messages.

3.2.3 Marketplace Browsing

- Introduction: Any logged-in user can browse the full product catalog and filter it by category.
- Inputs: An optional category parameter in the URL query string.
- Processing: We query the products table, optionally filtering by category, and eager-load the supplier relationship so we can show who is selling each item.

- **Outputs:** A responsive grid of product cards. Each card has the product image, name, category tag, description snippet, price, and current stock level.
- **Error Handling:** If there are no products matching the filter, we show a friendly empty state instead of a blank page.

3.2.4 Cart and Checkout

- **Introduction:** Users can build up a cart, adjust it, and then check out all at once.
- **Inputs:** Product ID when adding to cart, quantity when updating, and payment method when checking out.
- **Processing:** Cart state is stored in the server session as a simple product_id-to-quantity mapping. On checkout, we loop through the cart, create an Order record for each item, subtract from stock, and then clear the session.
- **Outputs:** The cart page shows a running total. After checkout, the user lands on their dashboard with a confirmation message.
- **Error Handling:** If someone tries to buy more than what is in stock, the validation catches it. If a product gets deleted while it is in someone's cart, the checkout just skips that item.

3.2.5 Order Management

- **Introduction:** When a purchase goes through, the system records it and updates inventory automatically.
- **Inputs:** Product ID, quantity, and chosen payment method.
- **Processing:** We create an order row with the buyer's user ID, the product ID, quantity, calculated total price, status set to pending, and the payment method. Then we decrement the product's stock.
- **Outputs:** A success message on the dashboard, plus the order shows up in the order history table.
- **Error Handling:** Stock validation runs before the order is created. If the product does not exist, Laravel returns a 404.

3.5 Non-Functional Requirements

3.5.1 Performance

We have a hard ceiling of 10 seconds per request on Vercel's Hobby plan, but in practice most pages load in under 3 seconds. Database queries are simple enough that they finish well under 500ms. The 3D scene targets 30+ FPS on machines with a discrete GPU; on integrated graphics it still works but might drop to around 20 FPS. Static files (CSS, JS, images) come from Vercel's global CDN, so those are fast everywhere.

3.5.2 Reliability

The app's uptime basically depends on Vercel and Neon. Both have pretty solid track records, and Vercel's serverless setup means we do not have a single server that can go down. Database writes for orders use Laravel's built-in error handling, and if the Neon connection drops, the user gets a clean error page rather than a broken half-rendered mess.

3.5.3 Availability

The app is live at <https://farmers-shop-main.vercel.app> around the clock. Vercel handles scaling automatically — if ten people hit the site at once, it just spins up more function instances. The one thing to watch out for is Neon's cold start: if nobody has used the app for a while, the database takes maybe a second or two to wake up on the first request.

3.5.4 Security

We have got the basics covered: HTTPS everywhere (we force the scheme in the service provider), bcrypt for password hashing with a cost of 12, CSRF tokens on every form, and SSL on the database connection. The APP_KEY and database password live in Vercel's encrypted env var store, not in the source code (though the DB host and username are in `vercel.json` since those are not sensitive). Role checks happen in the controller before any supplier-only action executes.

3.5.5 Maintainability

Laravel's MVC structure keeps things organized — models in one place, controllers in another, views in their own folder. Database changes go through migration files, so you can track schema history in Git. We use Composer for PHP packages and npm for JS stuff, which makes the dependency story pretty clean.

3.5.6 Portability

The core app is just a Laravel project, so you could deploy it on any hosting that supports PHP 8.5 and Postgres. The Vercel-specific bits (`vercel.json`, the `api/index.php` entry point, the `/tmp` path redirects) would need to be removed, but the actual application code is not tied to Vercel at all. On the frontend, anything running a recent version of Chrome, Firefox, Safari, or Edge will work.

3.7 Design Constraints

- Vercel's filesystem is read-only, so we redirect all of Laravel's writable paths (cache, compiled views, sessions, logs) to `/tmp`
- The total deployment bundle cannot exceed 300 MB — that is why we compressed the panorama image from 28 MB down to 1.5 MB
- Neon requires us to pass an explicit endpoint ID in the connection string because the PHP libpq on Vercel is too old for SNI
- We went with cookie-based sessions instead of file or database sessions since there is no persistent disk in serverless

- Product images need to be small (under 2 MB each) or loaded from external URLs to keep the bundle manageable

3.9 Other Requirements

- We had to add `URL::forceScheme('https')` in the `AppServiceProvider` because Vercel terminates SSL at the edge, and Laravel was generating `http://` links by default.
- All five of Laravel's bootstrap cache files (services, packages, config, routes, events) are pointed at `/tmp/cache/` instead of the default `bootstrap/cache/` directory.

4. Analysis Models

4.1 Data Flow Diagrams (DFD)

- Level 0 — Context Diagram

At the highest level, TerraMarket sits between two types of users. Suppliers feed product data into the system (name, price, category, stock, images). Farmers send browse requests and purchase orders. The system sends back product listings, order confirmations, and dashboard data. Underneath, there is a single PostgreSQL database that stores everything.

- Level 1 — Breaking it Down

Process 1 handles authentication — registration, login, logout, and profile edits. It reads and writes to the users table.

Process 2 is product management — suppliers create listings, and the system stores them in the products table.

Process 3 is the marketplace display — it pulls from the products table, applies any filters, and sends the results to the view layer.

Process 4 manages the cart — all in-memory (well, in-session), no database writes until checkout.

Process 5 is order processing — it reads the cart, creates order records, and updates product stock.

- How the tables relate to each other:

One user (supplier) can have many products.

One user (farmer) can have many orders.

One product can appear in many orders.

So we have two one-to-many relationships fanning out from the users table, and one from products to orders.

5. GitHub Link

Repository: <https://github.com/Aadi-110i/FarmersShop>

6. Deployed Link

Live site: <https://farmers-shop-main.vercel.app>

7. Client Approval Proof

[Attach client approval proof here]

8. Client Location Proof

[Attach client location proof here]

9. Transaction ID Proof

[Attach transaction id proof here]

10. Email Acknowledgement

[Attach email acknowledgement here]

11. GST No

[Attach gst no here]

Appendix A — Tech Stack

Layer	What we used
Backend	Laravel 12.58.0 running on PHP 8.5.2
Frontend	Blade templates styled with Tailwind CSS (loaded from CDN), plus Alpine.js for small interactive bits
3D Graphics	Three.js 0.184.0 with EffectComposer for post-processing (brightness, contrast, gamma correction)
Animations	GSAP 3.12.2 — handles the preloader text animation and page reveal sequence
Database	Neon PostgreSQL 16, serverless hosted in us-east-1
Bundler	Vite 7.0.7 with the laravel-vite-plugin
Hosting	Vercel serverless (vercel-php@0.9.0 runtime)
Auth	Laravel Breeze with cookie-based sessions
Version Control	Git + GitHub (github.com/Aadi-110i/FarmersShop)
ORM	Eloquent — Laravel's built-in ORM for database queries

Appendix B — Database Schema

- Users Table

Columns: id (auto-increment primary key), name, email (unique), email_verified_at, password (bcrypt hash), role (farmer or supplier), remember_token, created_at, updated_at

- Products Table

Columns: id, user_id (foreign key to users, cascades on delete), name, image_url (nullable), category (one of: seeds, fertilizers, tools, manures), description, price (decimal with 2 places), stock_quantity (integer), created_at, updated_at

- Orders Table

Columns: id, user_id (who bought it), product_id (what they bought), quantity, total_price, status (pending/completed/cancelled), payment_method (text field), created_at, updated_at